

ANEXO III — ENTREGABLES

Proyecto Integral: Edificios Inteligentes y Sostenibilidad Energética · Curso 2025-2026

La fecha límite para todos los entregables es el **29 de mayo de 2026**. Todos los entregables son **por equipo**.

Los entregables del proyecto son tres: el **vídeo de demostración**, la **documentación técnica** y el **código fuente**. Se describen a continuación por orden de importancia para la evaluación.

6. Entregables

6.1 Vídeo de demostración (obligatorio — máxima visibilidad)

Duración: 10-15 minutos

Audiencia objetivo: Público general no técnico en Big Data e IA.

El vídeo es el entregable más visible del proyecto. Cumple cuatro funciones simultáneas:

1. **Demostrar** la funcionalidad de la solución desarrollada por el equipo.
2. Servir como **pieza de portfolio profesional** para los alumnos (CV, LinkedIn).
3. **Promover el Curso de Especialización** del IES Simarro ante futuros alumnos, empresas e instituciones.
4. Mostrar la conexión del trabajo con el proyecto institucional **CENTINELA+**.

Estructura recomendada del video:

Bloque	Contenido	Duración aprox.
1 — Contexto	El problema real que resuelve la solución. ¿Por qué importa?	1-2 min
2 — Demostración	Funcionalidad en vivo: dashboard, chatbot, modelos, benchmark Big Data	6-9 min
3 — Impacto	¿Qué mejora? ¿Cuánto? Conexión con CENTINELA+ e IES Simarro	2-3 min
4 — Cierre	Tecnologías usadas (sin detalles técnicos), curso, IES Simarro	1 min

Aspectos que NO deben aparecer: código fuente, configuración de servidores, parámetros de modelos, errores de implementación, discusiones técnicas internas.

Calidad mínima requerida:

- Grabación clara de pantalla con resolución mínima 1080p.
- Voz en off clara y bien articulada.
- Subtítulos recomendados (accesibilidad).
- Edición básica que elimine pausas largas y errores.
- Formato de entrega: MP4 (H.264).

Opción avanzada — Versión en inglés con voice cloning (opcional)

De forma opcional, los equipos pueden generar una versión del vídeo en inglés utilizando un modelo TTS (*Text-to-Speech*) con **clonación de voz** (*voice cloning*), de modo que la voz que narra en inglés sea perceptualmente similar a la del narrador original en español.

Esta opción amplía el alcance como herramienta de portfolio internacional y añade un caso de uso práctico de IA generativa de audio al proyecto.

Herramientas open-source recomendadas:

- **Coqui TTS** (<https://github.com/coqui-ai/TTS>) — Síntesis de voz y clonación de voz open-source en Python. Soporte para clonación de voz en pocos segundos de audio de referencia.
- **F5-TTS** (<https://github.com/SWivid/F5-TTS>) — Modelo TTS con clonación de voz de alta calidad.
- **Bark** (<https://github.com/suno-ai/bark>) — Modelo generativo de audio que puede clonar voces a partir de ejemplos cortos.

Proceso sugerido:

1. Grabar el vídeo en español (versión principal).
2. Escribir el guion en inglés.
3. Usar el modelo TTS con voice cloning para generar el audio en inglés con la voz del narrador.
4. Sincronizar el audio generado con el vídeo (puede ajustarse la velocidad del vídeo si es necesario).

Opción avanzada — Versión con despliegue en AWS (opcional)

De forma opcional, los equipos pueden desarrollar una versión parcial de la solución funcionando en **AWS** (Amazon Web Services), como demostración de arquitecturas cloud y alternativa al despliegue on-premise.

Servicios AWS relevantes para el proyecto:

- **AWS IoT Core** — Alternativa a Mosquitto para el broker MQTT.
- **Amazon Timestream** — Base de datos de series temporales managed (alternativa a InfluxDB).
- **Amazon OpenSearch Service** — Elasticsearch managed (alternativa para el índice vectorial).
- **Amazon Bedrock** — Acceso a LLMs managed (alternativa a Ollama).
- **AWS SageMaker** — Entorno de entrenamiento y despliegue de modelos.

6.2 Documentación técnica (obligatoria)

6.2.1 Documento de distribución de tareas por equipo (obligatorio)

Dado que la nota del proyecto es **por equipo**, cada alumno debe poder demostrar su contribución al resultado colectivo. Para ello, **todos los equipos deben entregar un documento** que especifique:

- **Composición del equipo:** nombre de cada integrante.
- **Distribución de responsabilidades:** qué partes del caso de uso ha liderado cada persona (con detalle suficiente para que el profesor pueda asignar la nota de forma justificada).
- **Complejidad estimada de cada tarea:** una valoración por parte del propio equipo de la dificultad relativa de cada tarea (baja / media / alta), con justificación breve.
- **Contribuciones al benchmark Big Data (Caso I):** quién del equipo participó en la configuración del clúster y la ejecución del benchmark, aunque el equipo no sea el responsable principal del Caso I.

Nota: El profesor podrá usar este documento para ajustar la nota individual de algún alumno si se evidencia una contribución muy desequilibrada dentro del equipo. La nota de equipo es el punto de partida; el documento de tareas puede ser argumento de ajuste a la baja para quienes no hayan contribuido de forma real.

Formato: Documento de texto (PDF o Markdown), 1-2 páginas máximo.

6.2.2 Runbook de instalación y configuración

Instrucciones paso a paso para **reproducir completamente el entorno** del equipo desde cero:

- Sistema operativo y versión (ej: Ubuntu 22.04 LTS).
- Lista exacta de dependencias con versiones: requirements.txt para Python, package.json para Node, etc.
- Comandos de instalación exactos (copiables directamente en la terminal).
- Ficheros de configuración completos: telegraf.conf, mosquito.conf, docker-compose.yml, variables de entorno (.env.example).
- Instrucciones de configuración específicas de los servidores del ITI:
 - Para el equipo DevOps/MLOps: configuración del clúster Hadoop/Spark (nodos, HDFS, YARN).
 - Para el equipo del chatbot: configuración del servidor Ollama y ElasticSearch.
- Instrucciones de arranque y parada de todos los servicios.
- Sección de troubleshooting: errores comunes y cómo resolverlos.

i Criterio de éxito del runbook Una persona que no participó en el proyecto debe poder reproducir el entorno siguiendo el runbook sin hacer preguntas al equipo.

6.2.3 Notebooks documentados

Todos los Jupyter Notebooks utilizados deben cumplir:

- **Celdas Markdown explicativas** al inicio de cada sección que describan el objetivo, el razonamiento y las decisiones tomadas (no solo el código).
- **Orden explícito de ejecución:** numeración clara de notebooks si son múltiples (01_ingesta.ipynb, 02_exploratorio.ipynb, 03_modelo.ipynb...) y documentación de dependencias entre ellos.
- **Outputs incluidos:** los notebooks deben tener los resultados de la última ejecución visibles (tablas, gráficos, métricas) para que sean verificables sin re-ejecutarlos.
- **Registro en MLflow referenciado:** cualquier experimento de ML debe referenciar el identificador del run en MLflow correspondiente.
- **Comentarios sobre dificultades:** breve mención de los problemas encontrados y cómo se resolvieron — esto es pedagógicamente valioso y se evaluará positivamente.

6.2.4 Diagrama de arquitectura

- Diagrama completo del sistema del equipo (componentes, flujos de datos, protocolos de comunicación).
- Herramienta libre: draw.io, Lucidchart, Mermaid (para incluir en el README del repositorio).
- Decisiones arquitectónicas documentadas con su justificación (¿por qué InfluxDB y no PostgreSQL? ¿por qué LangChain y no LlamaIndex?).

6.3 Código fuente (obligatorio)

Repositorio Git (GitHub o GitLab del ITI) con la siguiente estructura:

Ruta	Contenido
README.md	Descripción, arquitectura, instrucciones de instalación y uso
docs/runbook.md	Runbook de instalación y configuración
docs/arquitectura.md	Diagrama y decisiones arquitectónicas
docs/distribucion_tareas.md	Documento de distribución de tareas por equipo
notebooks/	Jupyter Notebooks numerados (01_, 02_, 03_...)
src/	Código fuente organizado por módulo (ingestion/, models/, api/)
config/	Ficheros de configuración (telegraf.conf, mosquitto.conf, grafana/)
requirements.txt / environment.yml	Dependencias Python con versiones exactas
docker-compose.yml (si aplica)	Orquestación de servicios con Docker

```

repositorio/
├── README.md                # Descripción del proyecto,
                             # arquitectura, cómo empezar
├── docs/
│   ├── runbook.md          # Runbook de instalación
│   ├── arquitectura.md      # Diagrama y decisiones arquitectónicas
│   └── distribucion_tareas.md # Documento de distribución de tareas
├── data/
│   └── .gitkeep             # Los datos NO van en Git (solo la
                             # estructura)
├── notebooks/
│   ├── 01_ingesta.ipynb
│   ├── 02_exploratorio.ipynb
│   └── 03_modelo.ipynb
├── src/
│   ├── ingestion/
│   ├── models/
│   └── api/
├── config/
│   ├── telegraf.conf
│   ├── mosquitto.conf
│   └── grafana/
├── requirements.txt
├── environment.yml          # Conda environment (alternativo)
└── docker-compose.yml      # Si aplica

```

Buenas prácticas de código — PEP8 y Git

¿Qué es PEP8? PEP8 (Python Enhancement Proposal 8) es la guía de estilo oficial de Python, que define convenciones de formato y nomenclatura para mejorar la legibilidad y consistencia del código. Las reglas más importantes:

Regla PEP8	Descripción
Indentación	4 espacios por nivel de indentación (nunca tabuladores)
Longitud de línea	Máximo 79 caracteres (88 con el formateador Black)
Líneas en blanco	2 líneas en blanco entre funciones/clases de nivel superior; 1 línea en blanco entre métodos de una clase.
Importaciones	al inicio del fichero, ordenadas: primero librería estándar, luego terceros, luego módulos propios.
Nombres de variables	snake_case para variables y funciones; CamelCase para clases; UPPER_CASE para constantes

Regla PEP8	Descripción
Comentarios	En inglés o en español de forma consistente; describir el <i>porqué</i> , no el <i>qué</i> (el código ya dice el qué).

Se recomienda usar **black** (`pip install black`) como formateador automático y **flake8** (`pip install flake8`) como linter. Referencia: <https://pep8.org>

Historial de commits: descriptivo y atómico. Cada commit debe corresponder a un cambio cohesivo con un mensaje que explique qué se hizo y por qué. No usar mensajes genéricos como "fix", "changes" o "update".

Nota: Los datasets y los modelos de gran tamaño NO deben incluirse en el repositorio Git. Usar lakeFS para los datasets y MLflow para los artefactos de modelos.